

Machine Learning Challenges in Mechanical Engineering

A beginner-friendly tutorial on diagnosing messy data, stabilizing training, and communicating uncertainty

Audience	Mechanical Engineering students with beginner-level Python and introductory machine learning background
Focus	Data quality diagnostics, practical fixes, training stability, and honest uncertainty communication

Learning promise. By the end of this tutorial, students should be able to diagnose common data problems, apply simple fixes, recognize unstable training, and explain model outputs with appropriate caution in engineering decision-making.

Prepared as a self-contained teaching document
7 April 2026

How to use this tutorial and intended learning outcomes

This tutorial is written for Mechanical Engineering students who are new to data-quality diagnostics, model reliability, and uncertainty communication. It starts with intuition, moves to simple mathematics, and then shows short classroom-ready code examples. The examples use Mechanical Engineering settings such as rare-defect inspection, material strength prediction, structural health monitoring, environmental monitoring, and Mechanical System Reliability and Safety.

Intended learning outcomes. After working through the document, a beginner should be able to diagnose class imbalance, noisy data, missing values, outliers, and drift; apply simple remedies such as resampling, Synthetic Minority Over-sampling Technique (SMOTE), imputation, and outlier handling; stabilize training with learning-rate control, early stopping, and regularization; and communicate uncertainty using thresholds, calibration, confidence levels, and prediction bands.

What this tutorial teaches

- Why real Mechanical Engineering datasets are often messy, incomplete, imbalanced, and time-varying.
- How to connect common machine learning failure modes to practical engineering risks.
- How to read beginner-level Python code for data diagnostics and model evaluation.
- How to choose simple, transparent fixes before moving to more advanced methods.
- How to present probabilities and prediction intervals honestly to decision-makers.

Detailed outline and recommended teaching sequence

The document follows a beginner-first sequence. The sequence is deliberate: students first learn the minimum coding and machine learning language needed to read examples, then learn how to inspect data problems, and only after that learn how to stabilize training and communicate uncertainty. This order mirrors responsible engineering practice, where understanding the data comes before trusting a model.

- Stage 1: Build a common language. Learn variables, arrays, tables, features, targets, probabilities, residuals, and the roles of training, validation, and test data.
- Stage 2: Adopt a diagnostic mindset. Before training any model, ask where the data came from, whether the classes are balanced, whether values are missing or extreme, and whether the operating environment has changed.
- Stage 3: Study Part I on data issues and practical fixes. Work first on imbalance and noise, then on missing data and outliers, and finally on drift checks and deployment monitoring.

- Stage 4: Study Part II on training stability, risk, and uncertainty. Learn how unstable training appears, how learning-rate and regularization tools help, and how probabilities and intervals should be communicated in an engineering setting.
- Stage 5: Integrate the ideas through Mechanical Engineering mini-workflows and deployment checklists.

Teaching stage	Main question	Typical output
Mounting bases	What do the symbols, arrays, and evaluation words mean?	Students can read the examples without being lost.
Data diagnostics	Is the dataset trustworthy enough for modeling?	Students can spot imbalance, missingness, outliers, noise, and drift.
Training stability	Is the model learning in a stable and generalizable way?	Students can read training and validation curves and choose simple remedies.
Uncertainty communication	How should predictions be reported to support engineering decisions?	Students can explain thresholds, calibration, and intervals honestly.

Table of contents

The contents list below is intentionally static and easy to scan. The main body follows the same sequence.

How to use this tutorial and intended learning outcomes

What this tutorial teaches

Detailed outline and recommended teaching sequence

Table of contents

1. Introduction to machine learning challenges in Mechanical Engineering

1.1 Why real Mechanical Engineering data are difficult

1.2 Why these challenges matter for engineering decisions

1.3 Typical Mechanical Engineering applications where challenges appear

1.4 Section summary

2. Python and machine learning mounting bases for this tutorial

2.1 The smallest set of Python ideas you need

2.2 Features, labels, targets, predictions, probabilities, residuals, and thresholds

2.3 Training, validation, and test data

2.4 A short starter code example

2.5 Section summary

3. A diagnostic mindset before building a model

3.1 Questions to ask before fitting any model

3.2 A quick diagnostic checklist

3.3 Section summary

Part I. Data issues and practical fixes

4. Class imbalance and noise

- 4.1 Extreme class imbalance and rare-event detection
- 4.2 Why accuracy can be misleading
- 4.3 Oversampling, undersampling, SMOTE, and class weighting
- 4.4 Noisy labels and noisy measurements
- 4.5 Mechanical Engineering interpretation
- 4.6 Section summary

5. Missing data and outliers

- 5.1 Patterns of missing data
- 5.2 Simple imputation strategies
- 5.3 Outliers and extreme values
- 5.4 Clipping, winsorizing, and careful removal
- 5.5 Mechanical Engineering interpretation
- 5.6 Section summary

6. Data drift and deployment checks

- 6.1 What drift means in practice
- 6.2 Simple drift checks for beginners
- 6.3 Practical alert rules and retraining triggers
- 6.4 Mechanical Engineering interpretation
- 6.5 Section summary

7. Theory and mathematics for data-quality diagnostics

- 7.1 Imbalance ratios and confusion-matrix quantities
- 7.2 Imputation intuition
- 7.3 z-score and interquartile range rules
- 7.4 Distribution comparison and drift diagnostics
- 7.5 Section summary

Part II. Training stability, risk, and uncertainty communication

8. Unstable training

- 8.1 What unstable training looks like
- 8.2 Learning-rate problems
- 8.3 Batch size, initialization, and unstable validation
- 8.4 Overfitting and underfitting
- 8.5 Section summary

9. Stabilization tools

- 9.1 Learning-rate adjustment and scheduled reduction
- 9.2 Early stopping
- 9.3 Regularization and L2 penalties

- 9.4 Dropout and validation monitoring
- 9.5 A practical Keras template for stable training
- 9.6 Section summary

10. Uncertainty communication and risk-aware decision-making

- 10.1 Hard decisions versus confidence levels
- 10.2 Probability calibration and reliability diagrams
- 10.3 Decision thresholds and threshold-versus-metric thinking
- 10.4 Prediction intervals and residual-based error bands
- 10.5 Repeated-run uncertainty summaries
- 10.6 Communicating uncertainty honestly
- 10.7 Section summary

11. Theory and mathematics for training stability and uncertainty

- 11.1 Learning-rate effects on optimization
- 11.2 Generalization and overfitting intuition
- 11.3 Regularization penalties
- 11.4 Calibration intuition and threshold logic
- 11.5 Empirical prediction intervals and error bands
- 11.6 Section summary

12. Integrated Mechanical Engineering mini-workflows

- 12.1 Rare defect detection in mechanical systems inspection
- 12.2 Mechanical material strength prediction
- 12.3 Structural health monitoring from sensor data
- 12.4 Mechanical System Reliability and Safety: site and time effects
- 12.5 Deployment checklist
- 12.6 Section summary

13. Best practices and common mistakes

- 13.1 A practical checklist before, during, and after training
- 13.2 Common mistakes made by beginners
- 13.3 Section summary

14. Conclusion

References

1. Introduction to machine learning challenges in Mechanical Engineering

Machine learning is attractive in Mechanical Engineering because it can learn useful patterns from data that are too large, noisy, or nonlinear for simple hand-built formulas alone. Examples include predicting mechanical material strength from mixture properties, detecting cracks in images, classifying bearing failure susceptibility, forecasting water levels, screening mechanical component-inspection records, and tracking structural health monitoring signals over time. Reviews in structural health monitoring and broader Mechanical Engineering applications show that data-driven methods can support inspection, prediction, and decision-making across a wide range of mechanical systems problems [1], [2].

At the same time, real engineering data are rarely clean. Field measurements may be noisy, sensors may fail during high-load events, labels may be incomplete because only a small portion of structures are manually inspected in detail, and operating conditions may shift from site to site or season to season. This is especially important in applications linked to Mechanical System Reliability and Safety, where site effects, time effects, and extreme events can make the future look different from the data used to train the model [3], [4].

Core idea. A model can look successful in a notebook and still behave poorly in practice if the training data are imbalanced, missing, noisy, or no longer representative of the field conditions.

1.1 Why real Mechanical Engineering data are difficult

Mechanical Engineering data often come from physical systems exposed to weather, manufacturing variability, maintenance history, sensor drift, human inspection subjectivity, and changing mounting conditions. That means a model may have to work with measurements collected at different times, with different devices, under different loading conditions, and sometimes with imperfect ground truth labels. A turbine blade surface image taken in bright daylight may look very different from one taken in thermal cycling, and a vibration sensor can behave differently after maintenance or temperature change.

Mechanical Engineering task	Typical data source	Common machine learning challenge	Practical consequence
Rare defect detection	Inspection images or condition logs	Extreme class imbalance	High accuracy can hide missed defects.
Material strength prediction	Mixture design tables and lab tests	Missing values and outliers	Unstable predictions or biased fit.
Structural health monitoring	Strain, acceleration, displacement time series	Noise and drift over time	False alarms or missed anomalies.

Air-quality or environmental monitoring	Sensor streams	Calibration drift and missingness	Biased trend estimates.
Natural hazard classification	Topography, flow-rate, material, remote sensing	Shift across site and season	Poor transfer to new locations.

1.2 Why these challenges matter for engineering decisions

Engineering decisions are often asymmetric. Missing a serious defect can be much more costly than over-inspecting a harmless region. Overstating confidence in a cooling-system overload-depth prediction may lead to poor preparedness. Failing to notice drift in a structural monitoring system may create false confidence exactly when operating conditions have changed. In other words, machine learning in engineering is not only about prediction accuracy; it is also about reliability, transparency, and responsible communication.

This is why the tutorial treats data diagnostics, training stability, and uncertainty communication as one connected topic. A stable model trained on poor data is still risky. A flexible model with strong accuracy but poor calibration can still mislead. Good engineering practice requires a chain of trust: reliable inputs, stable training, and honest output interpretation.

1.3 Typical Mechanical Engineering applications where challenges appear

- Mechanical systems inspection: detecting rare cracks, spalling, corrosion, or machined-surface defect from images.
- Structural systems: identifying damage states or anomalies from sensor time series in mechanical components, test structures, or offshore systems.
- Materials: predicting compressive strength, slump, or durability from mixture variables measured in the laboratory or field.
- Thermofluid analysis and hazards: forecasting flow-rate-coolant flow response, reservoir inflow, cooling-system overload depth, or bearing failure susceptibility under changing conditions.
- Environmental and vehicle systems systems: monitoring air quality, production flow, travel time, or sensor health across sites and seasons.

Each of these settings contains its own version of messy data, unstable modeling conditions, and uncertainty that must be communicated carefully to engineers, asset managers, and public agencies [1]-[4].

1.4 Section summary

- Mechanical Engineering machine learning problems are often affected by data imbalance, missingness, noise, outliers, and time-varying conditions.
- These issues matter because engineering decisions can be safety-critical and cost-sensitive.
- The tutorial therefore begins with data-quality diagnostics before moving to training stability and uncertainty communication.

2. Python and machine learning mounting bases for this tutorial

Only a small amount of Python is needed to follow the examples in this document. The goal is not to turn the reader into a software engineer. The goal is to make the code readable enough that the machine learning ideas are clear.

2.1 The smallest set of Python ideas you need

Concept	Meaning in plain language	Typical example in this tutorial
Variable	A named object stored in memory	<code>water_cement_ratio = 0.48</code>
Array	A collection of values, often numeric	<code>X.shape = (n_samples, n_features)</code>
Table	Labeled columns, often using pandas	<code>df[['cement', 'water', 'strength']]</code>
Function	A reusable block of code	<code>def drift_flag(reference, current): ...</code>
Class	A blueprint for an object	<code>LogisticRegression(...)</code>
Method	A function attached to an object	<code>model.fit(X_train, y_train)</code>
Argument	An input given to a function or method	<code>learning_rate=0.001</code>

In machine learning code, arrays and tables appear constantly. For a tabular problem, the feature matrix is usually written as X and the target or label vector as y . If X has shape (500, 6), that means 500 observations and 6 features per observation. If y has shape (500,), there is one target value per observation.

2.2 Features, labels, targets, predictions, probabilities, residuals, and thresholds

A feature is an input variable used by the model. Examples in Mechanical Engineering include cement content, load intensity, mechanical component age, crack width, production throughput, or vibration amplitude. A target, also called a label in classification, is the quantity the model tries to predict. Examples include defect versus no defect, compressive strength, water level, or damage state.

For classification, the model often produces a probability, such as 0.83 for the event 'defect present'. A decision threshold turns that probability into a hard yes or no decision. For example, a threshold of 0.50 predicts defect when the probability is at least 0.50. A threshold of 0.80 is more conservative and may reduce false alarms but miss more true defects.

$$\text{residual } e_i = y_i - \hat{y}_i$$

For regression, the residual is the difference between the true value y_i and the prediction $\hat{y}_i$. Residuals are important because they show where the model systematically over-predicts or under-predicts.

2.3 Training, validation, and test data

The training set is used to fit the model parameters. The validation set is used to monitor performance while decisions are being made about hyperparameters, stopping time, or regularization. The test set is held back until the end and is used for an honest final evaluation. Beginners often make the mistake of repeatedly looking at the test set during development. That leaks information from testing into model design and makes the final score less trustworthy.

Practical rule. If the data contain a time order, do not randomly shuffle away the time structure without thinking. In monitoring and forecasting problems, older data often belong in training and newer data in validation or testing.

2.4 A short starter code example

Inputs and outputs. The table stores mixture variables and strength. X is a two-dimensional feature array, y is a one-dimensional target array, and the split produces smaller arrays for training and testing.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

# A small Mechanical Engineering-style table
df = pd.DataFrame(
    {
        "cement": [350, 370, 330, 390, 360],
        "water": [180, 175, 190, 170, 178],
        "age_days": [7, 14, 7, 28, 14],
        "strength_mpa": [28.5, 34.1, 24.7, 41.3, 33.0],
    }
)

# Features X and target y
X = df[["cement", "water", "age_days"]].to_numpy()
y = df["strength_mpa"].to_numpy()

print("X shape:", X.shape)    # (5, 3)
print("y shape:", y.shape)    # (5,)

# Split the data for model development
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.40, random_state=42
```

```
)

print("Training samples:", X_train.shape[0])
print("Test samples:", X_test.shape[0])
```

The code uses only the core ideas needed throughout the tutorial: a table, a feature matrix, a target vector, and a train-test split.

Simple plotting. A histogram is often the first diagnostic tool used to look for skewness, missing values, or unusual tails in a Mechanical Engineering variable.

```
import matplotlib.pyplot as plt

# Simple plotting example: inspect a feature distribution
plt.hist(df["strength_mpa"], bins=5)
plt.xlabel("Mechanical material strength (MPa)")
plt.ylabel("Count")
plt.title("A small strength distribution")
plt.show()
```

2.5 Section summary

- The tutorial uses only beginner-level Python ideas: variables, tables, arrays, functions, classes, methods, and arguments.
- X usually means the feature matrix and y usually means the target vector.
- Classification often produces probabilities and then uses a threshold to make decisions.
- Regression often uses residuals to study error behavior.
- Training, validation, and test sets have different roles and should not be mixed.

3. A diagnostic mindset before building a model

Before fitting a model, it helps to pause and ask basic diagnostic questions. In many student projects, the first model is trained before the data are inspected carefully. That is the reverse of good engineering practice. A simple diagnostic mindset can prevent many later problems.

3.1 Questions to ask before fitting any model

Question	Why it matters	Typical action
Where did the data come from?	Different sensors, sites, or inspection teams may create hidden bias.	Record source, site, and time metadata.
Is the target rare?	A rare class changes which metrics are trustworthy.	Check class counts before training.
Are values missing?	Missingness may be random or may signal a system problem.	Compute missing-value percentages and patterns.
Are there extreme values?	Outliers may be physical extremes or measurement errors.	Inspect summaries, box plots, and simple rules.

Has the environment changed?	A deployed model may face drift over time.	Compare recent data with historical reference data.
What happens if the model is wrong?	Engineering decisions have different risk levels.	Choose metrics and thresholds based on the consequence of error.

Engineering interpretation. In a safety-related application, the most important question is often not 'What is the highest accuracy?' but 'Which mistakes matter most, and how do we detect them early?'

3.2 A quick diagnostic checklist

- Check the units and physical meaning of every feature before computing statistics.
- Count observations by class, by site, and by time period.
- Measure the fraction of missing values in each feature.
- Inspect simple summaries such as mean, median, standard deviation, minimum, maximum, and percentiles.
- Look for unreasonable values based on engineering knowledge, not only statistical rules.
- Separate validation and test data before experimenting with fixes.
- Write down how you will explain probability, threshold choice, and uncertainty to a decision-maker.

3.3 Section summary

- A diagnostic mindset reduces the chance of trusting a model built on misunderstood data.
- Simple checks performed early can reveal imbalance, missingness, outliers, and drift.
- In Mechanical Engineering, error consequences should guide the choice of metrics and thresholds.

Part I. Data issues and practical fixes

Part I focuses on data problems that commonly appear before or during model development. The emphasis is practical: how to recognize the problem, why it matters in Mechanical Engineering, and which simple corrective actions are reasonable for a beginner to try first.

Theme	Main question	Typical beginner tools
Imbalance and noise	Is the target rare, and are the labels or measurements trustworthy?	Confusion matrix, precision, recall, resampling, SMOTE, class weighting
Missing data and outliers	Are important values absent or suspiciously extreme?	Missing-value summary, imputation, z-score, interquartile range, clipping
Drift checks	Does the new operating environment still look like the training environment?	Mean and variance comparison, percentiles, rolling statistics, alert flags

Teaching message. Fixing every data issue perfectly is rarely possible. The goal is to identify the dominant problem, apply a transparent remedy, and document the remaining uncertainty.

4. Class imbalance and noise

Class imbalance means that one class is much more common than another. In Mechanical Engineering, this is very common because severe events are often rare. A dataset may contain 99 percent 'no defect' images and only 1 percent 'defect' images. The same pattern appears in rare bearing failure events, rare structural damage states, and rare emergency incidents [6]-[8].

Noise is a related but different problem. Noise means that the input measurements or the labels are imperfect. A crack label may be wrong because two inspectors disagree. A sensor measurement may be noisy because of temperature effects or poor calibration. Imbalance makes rare events hard to learn, while noise makes the available signal less trustworthy.

4.1 Extreme class imbalance and rare-event detection

Plain-language explanation. In a strongly imbalanced problem, the model sees many examples of the majority class and only a few examples of the minority class. Purpose of diagnosing imbalance. The aim is to understand whether the model is learning the rare event at all, rather than simply becoming good at predicting the common class.

Inputs and outputs. The input is a labeled dataset with class counts. The output of the diagnostic step is a summary of class frequencies, imbalance ratio, and metrics that focus on the minority class. In a defect-detection problem, the important question is usually how many true defects are found and how many are missed.

$$\text{imbalance ratio} = N_{\text{majority}} / N_{\text{minority}}$$

Why it matters in Mechanical Engineering. If the minority class corresponds to a rare but serious defect, a model that misses most positives can still appear impressive if judged only by accuracy. That is unacceptable in many engineering screening tasks.

4.2 Why accuracy can be misleading

Suppose a machined surface-inspection dataset has 9,900 images labeled no-distress and 100 images labeled distress. A classifier that predicts no-distress for every image achieves 99 percent accuracy, but it completely fails at the real task. This is why confusion matrices, precision, recall, and the F1-score are essential under imbalance [7], [8].

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

Figure 1 is useful because it forces the analyst to separate the different kinds of mistakes instead of compressing everything into one accuracy value.

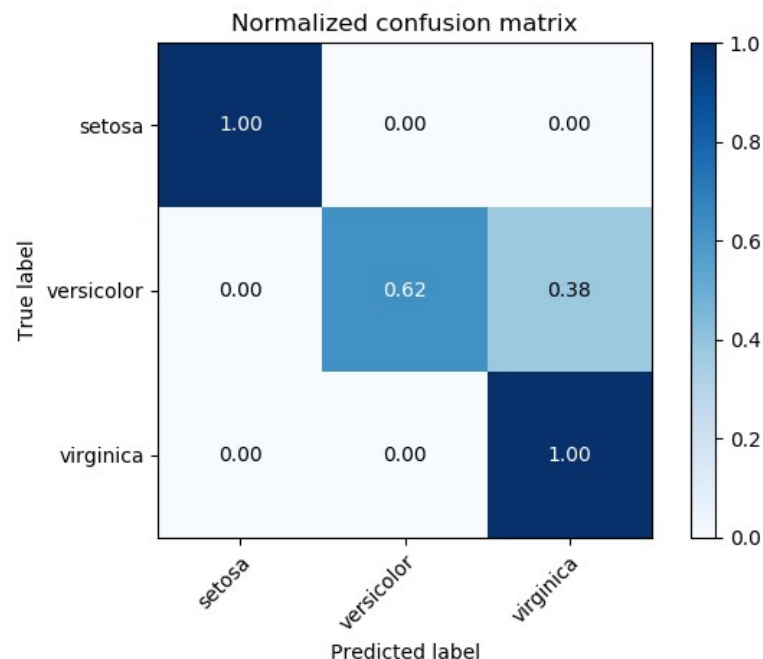


Figure 1. Example normalized confusion matrix. In imbalanced classification, the confusion matrix is a better diagnostic tool than raw accuracy because it shows true positives, false negatives, false positives, and true negatives separately.

Source: scikit-learn developers [18].

Figure 2 reminds us that a classifier can trade precision against recall as the decision threshold changes. Under severe imbalance, precision-recall thinking is often more informative than raw accuracy, and in many cases it is also more revealing than receiver operating characteristic (ROC) summaries because the rare positive class deserves direct attention [8], [19].

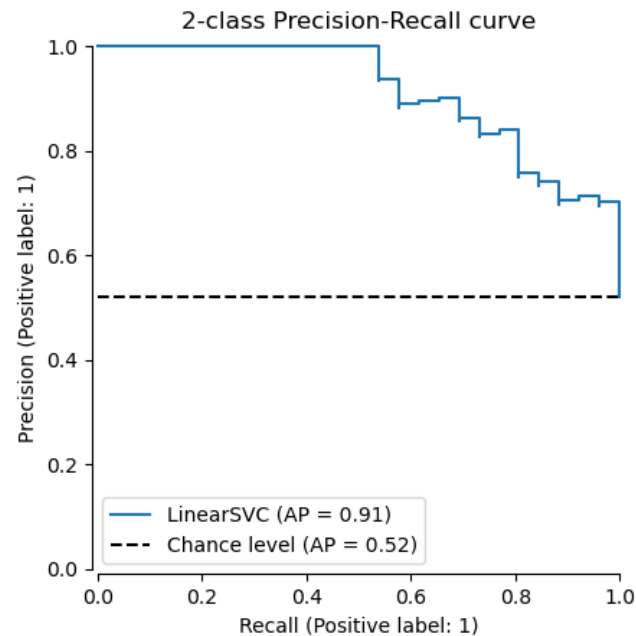


Figure 2. Precision-recall curve. For strongly imbalanced binary problems, precision and recall often provide a clearer picture of rare-event detection quality than accuracy alone. Source: scikit-learn developers [19].

Metric	What it answers	Use under imbalance
Accuracy	How many predictions were correct overall?	Can be misleading when the majority class dominates.
Precision	Of predicted positives, how many were truly positive?	Useful when false alarms are expensive.
Recall	Of true positives, how many were detected?	Useful when missing a rare event is costly.
F1-score	How well are precision and recall balanced?	Useful as a single summary when both matter.
Confusion matrix	Which errors occurred and how often?	Essential diagnostic view before trusting any summary metric.

Inputs and outputs. X has shape (5000, 8), y is a binary label vector, and the code prints a confusion matrix and detailed metrics on the validation set.

```
import numpy as np
```

```

from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split

# Synthetic rare-defect dataset: 99% no defect, 1% defect
X, y = make_classification(
    n_samples=5000,
    n_features=8,
    n_informative=4,
    n_redundant=0,
    weights=[0.99, 0.01],
    flip_y=0.0,
    random_state=42,
)

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_valid)
print(confusion_matrix(y_valid, y_pred))
print(classification_report(y_valid, y_pred, digits=3))

```

A beginner should read the output in this order: recall of the minority class, precision of the minority class, and then the overall picture in the confusion matrix.

4.3 Oversampling, undersampling, SMOTE, and class weighting

Oversampling repeats or synthesizes minority-class observations so that the model sees them more often during training. Undersampling removes part of the majority class. Synthetic Minority Over-sampling Technique, usually abbreviated as SMOTE, creates new minority examples by interpolating between nearby minority points [6], [15], [20]. Class weighting keeps the original data but makes mistakes on the minority class count more heavily in the loss function.

Inputs and outputs. The input is the original training data and, for class weighting, a set of class weights such as a larger weight for the defect class. The output is a modified training process that is more sensitive to rare events. Each approach changes the training signal in a different way.

Figure 3 shows the geometric idea behind SMOTE. It does not simply duplicate rare samples. Instead, it creates new samples between nearby minority examples.

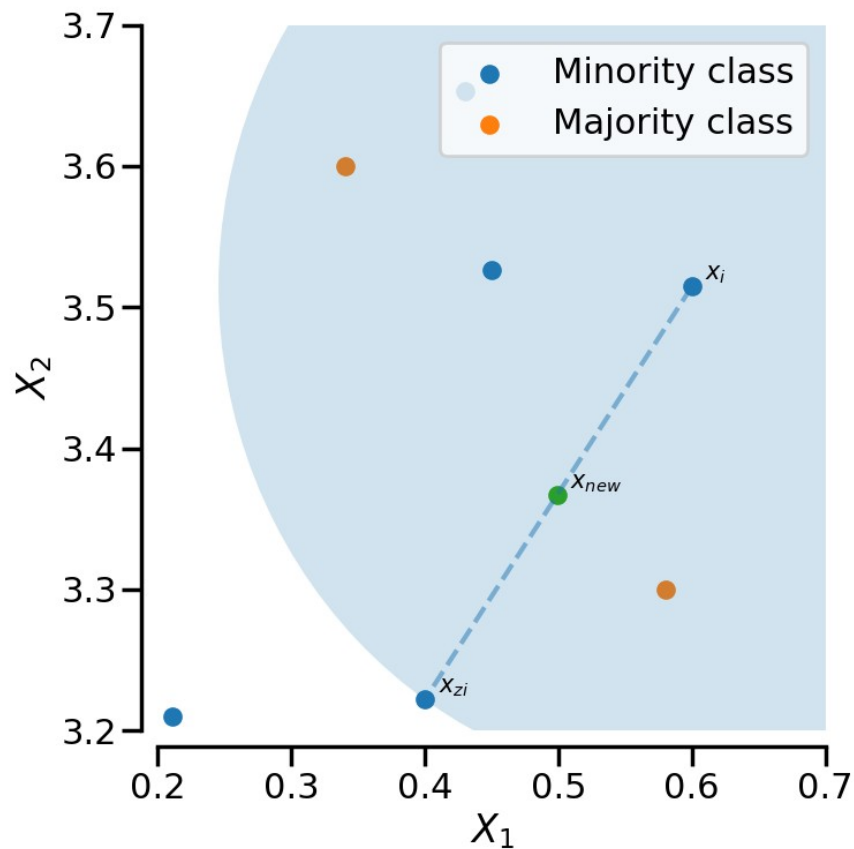


Figure 3. Illustration of Synthetic Minority Over-sampling Technique (SMOTE) sample generation. New minority samples are created along line segments between nearby minority points. Source: imbalanced-learn developers [20].

Remedy	Purpose	Strength	Caution
Random oversampling	Increase minority exposure	Simple and easy to explain	May repeat the same noisy minority samples many times.
Random undersampling	Reduce majority dominance	Fast and simple	Can discard useful majority information.
SMOTE	Create synthetic minority samples	Often stronger than naive duplication	Can create unrealistic points if minority regions are poorly defined.
Class weighting	Penalize minority errors more strongly	Keeps the original dataset unchanged	May not be enough if the class separation is extremely difficult.

Inputs and outputs. Each method takes the original training set and produces a modified training set. The validation set is left unchanged so the comparison stays honest.


```

from imblearn.over_sampling import RandomOverSampler, SMOTE
from imblearn.under_sampling import RandomUnderSampler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

# Try three simple remedies on the training data only
over = RandomOverSampler(random_state=42)
X_over, y_over = over.fit_resample(X_train, y_train)

under = RandomUnderSampler(random_state=42)
X_under, y_under = under.fit_resample(X_train, y_train)

smote = SMOTE(random_state=42)
X_smote, y_smote = smote.fit_resample(X_train, y_train)

for name, X_res, y_res in [
    ("oversample", X_over, y_over),
    ("undersample", X_under, y_under),
    ("smote", X_smote, y_smote),
]:
    model = LogisticRegression(max_iter=1000)
    model.fit(X_res, y_res)
    pred = model.predict(X_valid)
    print(name, "F1 =", round(f1_score(y_valid, pred), 3))

```

Best practice. Apply resampling only to the training data, never to the full dataset before the split.

Class weighting example. The input is the original training data and a weighting rule. The output is a model trained to pay more attention to the minority class.

```

weighted_model = LogisticRegression(
    max_iter=1000,
    class_weight="balanced"
)
weighted_model.fit(X_train, y_train)

weighted_pred = weighted_model.predict(X_valid)
print(classification_report(y_valid, weighted_pred, digits=3))

```

4.4 Noisy labels and noisy measurements

Noisy labels mean the stated target is sometimes wrong. Noisy measurements mean the inputs themselves are corrupted. Both reduce the reliability of the learning signal. For example, two inspectors may disagree about whether a hairline surface mark is a crack, or a strain gauge may have temporary interference from environmental conditions.

Simple mathematics. If a fraction of labels is flipped incorrectly, the model is partly trained on contradictions. If sensor measurements contain extra random variation, the same physical state can produce different input values. That usually lowers the best achievable performance and can make training curves noisier.

The code simulates label noise by corrupting a portion of the rare positive labels in the training data.

```

rng = np.random.default_rng(42)

# Flip 10% of the minority labels in the training set
y_train_noisy = y_train.copy()
minority_index = np.where(y_train == 1)[0]

```

```

flip_count = int(0.10 * len(minority_index))
flip_index = rng.choice(minority_index, size=flip_count, replace=False)
y_train_noisy[flip_index] = 0

noisy_model = LogisticRegression(max_iter=1000)
noisy_model.fit(X_train, y_train_noisy)

noisy_pred = noisy_model.predict(X_valid)
print(classification_report(y_valid, noisy_pred, digits=3))

```

Interpretation. If recall drops sharply after label corruption, the task is sensitive to annotation quality.

4.5 Mechanical Engineering interpretation

In mechanical systems inspection, the minority class often corresponds to the cases that matter most. Missing a real defect can delay repair or create false confidence in an asset condition report. In bearing failure or cooling-system overload-event classification, the rare event may be the event that drives emergency planning. That is why precision, recall, and threshold choice should be connected to the field consequence of each error type, not treated as abstract statistics.

Practical decision rule. If missing a positive event is much more costly than issuing a false alarm, optimize first for recall and then manage precision using threshold choice, post-screening, or human review.

4.6 Section summary

- Extreme class imbalance is common in Mechanical Engineering because serious events are often rare.
- Accuracy alone can be misleading; use confusion matrices, precision, recall, and F1-score.
- Oversampling, undersampling, SMOTE, and class weighting each change the training signal in a different way.
- Noise in labels or measurements can reduce performance and make rare-event learning even harder.

5. Missing data and outliers

Missing data and outliers are common in field and laboratory datasets. A value may be missing because a sensor failed, a technician skipped a measurement, or a record was not available at the time of collection. An outlier may be a genuine extreme event, a unit-conversion mistake, a sensor spike, or a data-entry error. The challenge is that the model itself usually does not know which explanation is correct.

5.1 Patterns of missing data

Plain-language explanation. Not all missing data are the same. Missing completely at random means the absence is unrelated to the observed or unobserved data. Missing at random

means the missingness can be partly explained by other observed variables. In practice, Mechanical Engineering datasets also show structured missingness, where missing values follow operational patterns such as sensor outages during high-load events, site-specific gaps, or missing laboratory variables for certain material batches [5], [17].

Why it matters. If missingness is structured, then simply pretending the data are complete can bias the model. For example, flow-rate sensors may fail during the most severe events, which means the missing values themselves carry information about stress on the monitoring system.

Figure 4 is useful because it shows that the pattern of missingness can itself contain information.

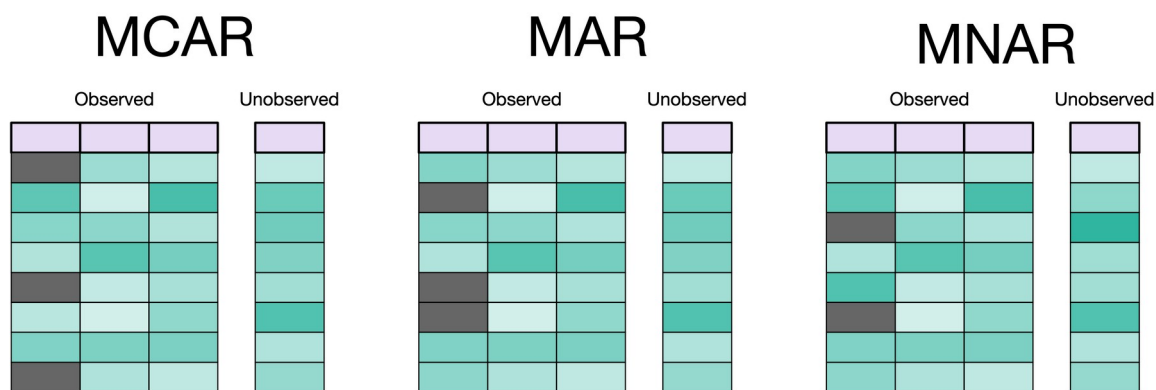


Figure 4. Missing-data mechanisms: missing completely at random (MCAR), missing at random (MAR), and missing not at random (MNAR). Source: Feature Engineering and Selection [17].

Inputs and outputs. The input is a table containing NaN values. The output is an imputed feature array with the missing entries replaced by the column means.

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer

df = pd.DataFrame(
    {
        "cement": [350, 370, np.nan, 390, 360],
        "water": [180, 175, 190, np.nan, 178],
        "curing_temp": [23.0, 22.5, 24.1, 23.7, np.nan],
        "strength_mpa": [28.5, 34.1, 24.7, 41.3, 33.0],
    }
)

print(df.isna().mean())  # fraction of missing values in each column

# Mean imputation for features
imputer = SimpleImputer(strategy="mean")
X_imputed = imputer.fit_transform(df[["cement", "water", "curing_temp"]])
print(X_imputed)
```

A first diagnostic step is simply to compute the missing-value fraction in each column and ask whether the missingness pattern is random or operationally meaningful.

5.2 Simple imputation strategies

Purpose. Imputation fills in missing entries so that a model can be trained. Mean imputation replaces missing values with the average, median imputation uses the middle value, and forward fill copies the most recent earlier value in a time series. **Inputs and outputs.** The input is a variable with missing entries. The output is a completed variable, together with a documented rule for how those replacements were chosen.

mean imputation: $x_{\text{missing}} \leftarrow \text{mean}(x_{\text{observed}})$

median imputation: $x_{\text{missing}} \leftarrow \text{median}(x_{\text{observed}})$

Simple mathematics and interpretation. Mean imputation is sensitive to extreme values, while median imputation is more robust when the variable is skewed. Forward fill can be useful for slowly varying sensor data, but it should be used carefully because it assumes the recent past is a good estimate of the present.

Method	Good first use case	Main strength	Main caution
Mean imputation	Roughly symmetric numeric feature	Very simple	Pulled by extreme values.
Median imputation	Skewed numeric feature	More robust than the mean	Still hides missingness structure.
Forward fill	Ordered time series with short gaps	Keeps continuity	Can be misleading across long gaps or abrupt events.

Inputs and outputs. The input is an ordered series with short missing runs. The output is a filled series where each gap is replaced by the last observed value.

```
# Forward fill for an ordered monitoring series
sensor = pd.DataFrame(
    {
        "hour": [1, 2, 3, 4, 5],
        "water_level_m": [2.10, np.nan, np.nan, 2.25, 2.30],
    }
)

sensor["water_level_ffill"] = sensor["water_level_m"].ffill()
print(sensor)
```

5.3 Outliers and extreme values

Plain-language explanation. An outlier is a value that is unusually far from the rest of the data. Some outliers are important physical extremes and should be kept. Others are errors. The goal is not to delete every unusual value. The goal is to determine whether the extreme value is plausible in the engineering context.

$$z\text{-score} = (x - \mu) / \sigma$$

$$IQR = Q3 - Q1$$

flag if $x < Q1 - 1.5 \times IQR$ or $x > Q3 + 1.5 \times IQR$

Why it matters. Outliers can shift the fit of a model, inflate error measures, and distort imputation. Z-scores compare a value to the mean and standard deviation, while the interquartile range (IQR) rule uses percentiles and is often more robust when the data are skewed.

Figure 5 illustrates how a few corrupted observations can bend a standard fit away from the main pattern.

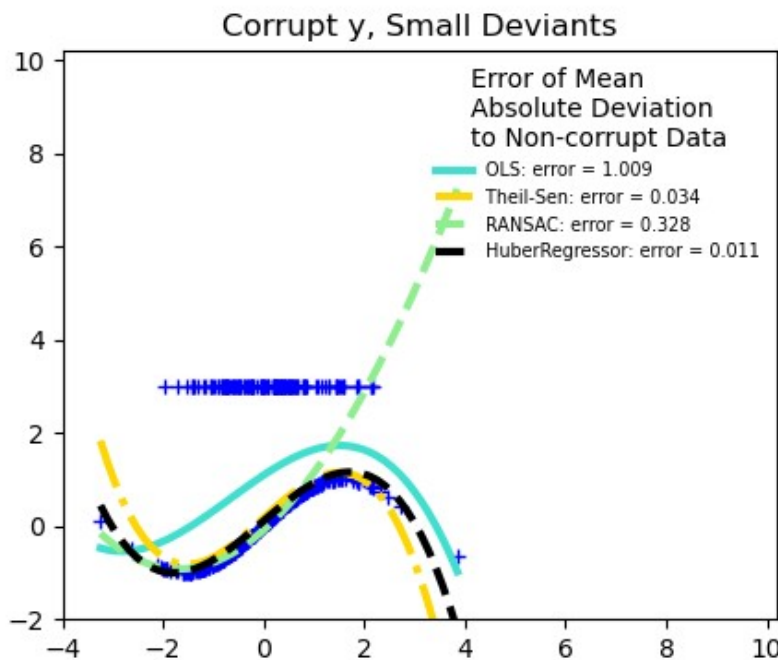


Figure 5. Robust fitting under corrupted observations. Outliers can pull a standard fit away from the main data pattern, which is why outlier diagnostics matter before trusting a model. Source: scikit-learn developers [21].

The code shows two common beginner-friendly outlier rules. The outputs are Boolean masks indicating which observations are flagged.

```
import numpy as np

strength = df["strength_mpa"].to_numpy()
z = (strength - strength.mean()) / strength.std(ddof=0)
outlier_mask_z = np.abs(z) > 3

q1, q3 = np.percentile(strength, [25, 75])
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outlier_mask_iqr = (strength < lower) | (strength > upper)

print("z-score flags:", outlier_mask_z)
print("IQR flags:", outlier_mask_iqr)
```

5.4 Clipping, winsorizing, and careful removal

Clipping means limiting extreme values to a chosen lower and upper bound. Winsorizing is a related idea in which values beyond selected percentiles are pulled back to those percentile limits. Removal means dropping flagged observations entirely. These actions are not interchangeable. Clipping keeps the observation but reduces its influence, while removal discards it and should only be done with clear justification.

Inputs and outputs. The input is a variable plus a rule such as percentile bounds or an IQR threshold. The output is a modified version of the variable or a filtered dataset. A responsible workflow always compares model performance before and after the change.

The first line creates a winsorized version of the target. The second example removes IQR-flagged observations, but only after the analyst has judged that the flags correspond to errors rather than meaningful extremes.

```
# Percentile-based winsorizing by clipping to the 1st and 99th percentiles
lower_p, upper_p = df["strength_mpa"].quantile([0.01, 0.99])
df["strength_winsor"] = df["strength_mpa"].clip(lower_p, upper_p)

# Removal with caution
clean_df = df.loc[~outlier_mask_iqr].copy()
```

Performance comparison. The purpose is not to prove that winsorizing is always better. The purpose is to document how the model behaves before and after remediation.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split

X = df[["cement", "water", "curing_temp"]].fillna(df.mean(numeric_only=True))
y_raw = df["strength_mpa"]
y_winsor = df["strength_winsor"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y_raw, test_size=0.40, random_state=42
)

m1 = LinearRegression().fit(X_train, y_train)
mae_raw = mean_absolute_error(y_test, m1.predict(X_test))

X_train, X_test, y_train, y_test = train_test_split(
    X, y_winsor, test_size=0.40, random_state=42
)

m2 = LinearRegression().fit(X_train, y_train)
mae_winsor = mean_absolute_error(y_test, m2.predict(X_test))

print("MAE with raw target:", round(mae_raw, 3))
print("MAE with winsorized target:", round(mae_winsor, 3))
```

Outlier action	When it is often useful	Engineering caution
Keep as is	Plausible extreme physical event	Do not erase important hazard or damage evidence.

Clip or winsorize	Probably real but overly influential extremes	Document the chosen bounds clearly.
Remove	Confirmed error or impossible value	Record the reason for deletion.

5.5 Mechanical Engineering interpretation

In a material-strength dataset, an outlier may come from a true but unusual mixture or from a recording mistake in water content. In structural monitoring, a sudden spike may reflect a sensor glitch or a real transient response. The statistical rule can only flag the value; engineering judgment is required to interpret it. Missing data should be treated with the same caution. A missing value may be harmless, or it may indicate a failure mode in the measurement system itself.

5.6 Section summary

- Missing data should be diagnosed before being filled. Missing completely at random is different from structured missingness.
- Mean, median, and forward fill are simple beginner-friendly imputation tools with different assumptions.
- Outliers require both statistical rules and engineering judgment.
- Clipping, winsorizing, and removal should be compared against baseline model performance and documented clearly.

6. Data drift and deployment checks

Data drift means that the distribution of the data seen during deployment is changing relative to the historical data used during development. This can happen because the environment changes, the sensor is recalibrated, the site is different, the season changes, or the inspection process is modified. Drift does not automatically mean the model has failed, but it is a warning that the model is operating farther away from the conditions it learned from [2]-[4].

6.1 What drift means in practice

Plain-language explanation. If the model was trained on one distribution and is later used on a different distribution, its assumptions may no longer fit the current data. For example, a river-level model trained mostly on normal flow conditions may later see more intense flow-rate seasons. An image-based defect model trained with one camera setup may later receive darker, blurrier, or differently scaled images.

Inputs and outputs. The input to a drift check is usually a reference dataset, often historical training data, and a current dataset, often recent observations. The output is a summary of how much the distributions differ and whether a flag should be raised for review or retraining.

6.2 Simple drift checks for beginners

Beginner-friendly drift checks include comparing the mean and standard deviation, comparing percentiles such as the 10th, 50th, and 90th percentiles, plotting histograms, tracking rolling statistics over time, and using simple alert rules. A more formal two-sample comparison is the Kolmogorov-Smirnov statistic, which measures the largest gap between the empirical cumulative distributions of two samples.

Figure 6 provides a visual picture of drift as a shift between an older reference distribution and a newer operating distribution.

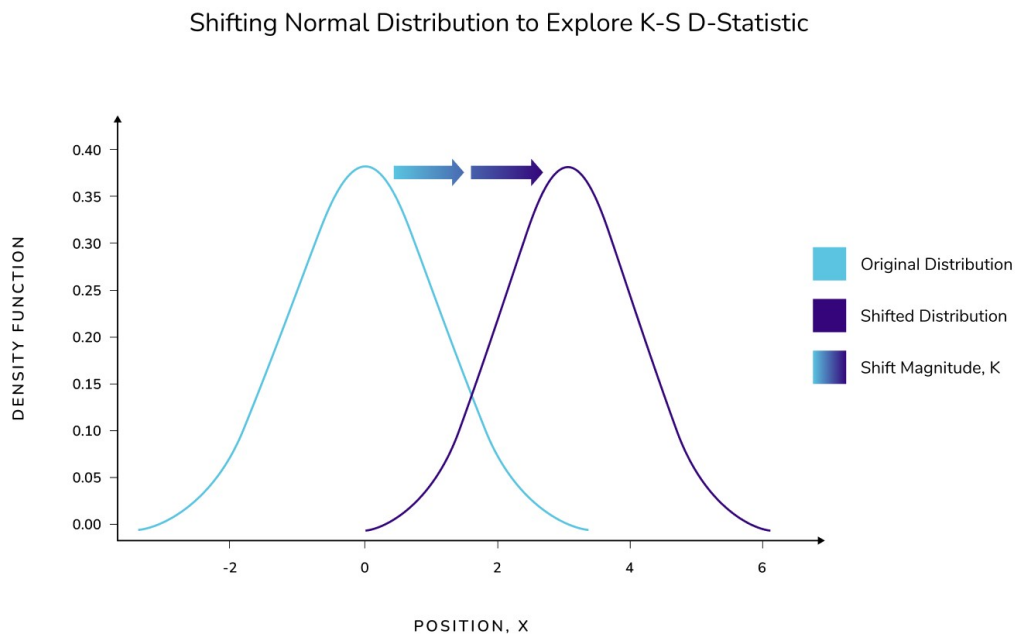


Figure 6. A distribution-shift illustration using the Kolmogorov-Smirnov idea. Drift checks compare how much the newer distribution has moved relative to the historical reference distribution. Source: NannyML [26].

Inputs and outputs. The input is a historical reference series and a newer current series. The output is a table of simple summaries plus a Kolmogorov-Smirnov drift statistic.

```
import numpy as np
import pandas as pd
from scipy.stats import ks_2samp

# Example: compare historical and recent water-level measurements
reference = pd.Series([2.0, 2.1, 2.3, 2.2, 2.1, 2.4, 2.2, 2.3])
current = pd.Series([2.4, 2.5, 2.6, 2.7, 2.6, 2.5, 2.8, 2.7])

summary = pd.DataFrame(
    {
        "reference": [
            reference.mean(),
            reference.std(ddof=0),
            np.percentile(reference, 10),
            np.percentile(reference, 50),
            np.percentile(reference, 90),
        ],
    },
)
```



```

        "current": [
            current.mean(),
            current.std(ddof=0),
            np.percentile(current, 10),
            np.percentile(current, 50),
            np.percentile(current, 90),
        ],
    },
    index=["mean", "std", "p10", "p50", "p90"],
)

ks_stat, ks_p = ks_2samp(reference, current)
print(summary)
print("KS statistic:", round(ks_stat, 3))
print("p-value:", round(ks_p, 3))

```

The code is intentionally simple. Many real systems monitor several features at once, but the same idea applies feature by feature.

Rolling statistics are useful when drift appears gradually over time rather than as one sudden jump.

```

# Rolling statistics for a time-ordered monitoring variable
series = pd.Series([2.10, 2.14, 2.12, 2.16, 2.18, 2.23, 2.27, 2.31])
rolling_mean = series.rolling(window=3).mean()
rolling_std = series.rolling(window=3).std()

print("rolling mean")
print(rolling_mean)
print("rolling std")
print(rolling_std)

```

6.3 Practical alert rules and retraining triggers

A simple drift flag can be defined using thresholds on summary statistics. For example, flag a feature if the mean changes by more than 10 percent of the historical mean, if the standard deviation ratio exceeds a chosen bound, or if a two-sample statistic exceeds a selected level. These rules are not universal. They should be chosen to match the sensitivity of the engineering application.

This alert rule is deliberately simple. It is not a universal standard, but it is a transparent starting point for a beginner project.

```

def drift_flag(reference, current):
    mean_shift = abs(current.mean() - reference.mean())
    mean_ratio = mean_shift / max(abs(reference.mean()), 1e-6)
    std_ratio = current.std(ddof=0) / max(reference.std(ddof=0), 1e-6)

    return {
        "mean_ratio": round(mean_ratio, 3),
        "std_ratio": round(std_ratio, 3),
        "flag": (mean_ratio > 0.10) or (std_ratio > 1.25),
    }

print(drift_flag(reference, current))

```

Practical retraining trigger. Do not retrain only because one feature moved slightly. A stronger trigger is the combination of measurable drift and model-performance decay on recent

labeled data. In deployed systems, this usually means monitoring both the incoming feature distribution and a delayed measure of actual performance whenever new labels become available.

Drift check	What it compares	Main strength	Main limitation
Mean and variance	Center and spread	Fast and easy to explain	May miss shape changes.
Percentiles	Distribution landmarks	Robust and interpretable	Still summarizes only part of the distribution.
Histogram overlap	Visual shape comparison	Good for communication	Depends on bin choice.
Rolling statistics	Time-local changes	Useful for gradual drift	Needs ordered data.
Kolmogorov-Smirnov idea	Maximum distribution gap	More formal than simple summaries	Still does not explain the cause of drift.

6.4 Mechanical Engineering interpretation

Drift is especially important when models are transferred from one site to another or from one time period to another. In Mechanical System Reliability and Safety, site and time effects are central. A cooling-system overload-related model trained on one coolant network or one flow-rate regime may not behave the same way in another. In structural monitoring, seasonal temperature changes can alter the baseline sensor pattern without indicating damage. Drift checks help separate environmental change from actual deterioration.

6.5 Section summary

- Drift means the current operating data no longer look like the historical reference data.
- Beginner-friendly drift checks include mean, variance, percentiles, histograms, rolling statistics, and simple alert rules.
- Retraining decisions should consider both distribution shift and recent model-performance decay.
- In Mechanical Engineering, drift often reflects site effects, time effects, sensor changes, or changing hazard conditions.

7. Theory and mathematics for data-quality diagnostics

This section gathers the main mathematical ideas from Part I into one place. The mathematics is intentionally light and is meant to support interpretation rather than replace engineering judgment.

7.1 Imbalance ratios and confusion-matrix quantities

If N_{majority} is much larger than N_{minority} , the minority class contributes only a small fraction of the total observations. In that situation, overall accuracy can remain high even while recall of the minority class stays poor. The confusion matrix separates the four basic outcomes: true positive (TP), false positive (FP), true negative (TN), and false negative (FN).

$$\text{Accuracy} = (TP + TN) / (TP + FP + TN + FN)$$

$$\text{Recall} = TP / (TP + FN)$$

$$\text{Precision} = TP / (TP + FP)$$

7.2 Imputation intuition

Mean imputation replaces each missing value with the average of the observed values. Median imputation replaces it with the middle observed value. These methods reduce the immediate problem of missingness but also reduce natural variability in the affected feature. That means they can make the data look more certain than they really are if used without caution.

7.3 z-score and interquartile range rules

The z-score measures how many standard deviations a value is from the mean. The IQR rule instead uses percentiles. Because the IQR depends on the central 50 percent of the data, it is often less sensitive to a few extreme values than the mean and standard deviation.

7.4 Distribution comparison and drift diagnostics

A distribution comparison asks whether two sets of values appear to come from similar operating conditions. A mean shift indicates a change in central tendency. A variance change indicates a change in spread. Percentile shifts indicate changes in the tails or in the median. None of these comparisons alone explains the cause of drift, but together they provide an interpretable early-warning system.

Quantity	Formula or rule	Interpretation
Imbalance ratio	$N_{\text{majority}} / N_{\text{minority}}$	Larger values mean the rare class is harder to learn from directly.
z-score	$(x - \mu) / \sigma$	Large magnitude means the value is far from the mean.
IQR	$Q3 - Q1$	Measures the spread of the central half of the data.
IQR outlier rule	x outside $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$	Flags unusual values based on percentiles rather than the mean.
Mean shift	$\text{mean}(\text{current}) -$	Detects central movement over time or

	mean(reference)	across sites.
Percentile shift	$p_k(\text{current}) - p_k(\text{reference})$	Detects changes in specific parts of the distribution.

7.5 Section summary

- The main formulas of Part I are simple and interpretable.
- Metrics under imbalance focus on what happens to the minority class.
- Imputation and outlier rules should be read as tools for diagnosis, not as automatic truth detectors.
- Drift diagnostics compare current data with a reference distribution to detect meaningful change.

Part II. Training stability, risk, and uncertainty communication

Part II moves from data preparation to model behavior during and after training. Even with a cleaned dataset, training can still be unstable, overly sensitive, or overconfident. This part explains how to diagnose unstable learning, apply simple stabilization tools, and report uncertainty honestly enough for engineering use.

Theme	Main practical question	Typical beginner tools
Unstable training	Is the model learning smoothly or behaving erratically?	Training curves, validation curves, learning-rate checks
Stabilization tools	Which simple actions make learning more stable and generalizable?	Early stopping, learning-rate reduction, L2 regularization, dropout
Uncertainty communication	How should prediction confidence be explained to decision-makers?	Thresholds, calibration plots, confidence histograms, prediction bands

8. Unstable training

Unstable training means the model parameters do not settle into a useful solution smoothly. The loss may diverge, oscillate, or decrease so slowly that little is learned. Validation performance may jump around or improve briefly and then worsen. These behaviors are common when the learning rate is poorly chosen, the model is too complex for the data, the batch size is awkward, or the inputs are noisy.

8.1 What unstable training looks like

Plain-language explanation. Training stability is judged by the behavior of the optimization process over epochs. A stable process usually shows a clear downward trend in training loss and a validation trend that improves and then levels off. An unstable process may show sudden spikes, wide oscillations, or divergence to very large losses. Inputs and outputs. The input is the model, the training data, and the hyperparameters. The output is a history of losses and metrics over epochs.

Figure 7 provides a clean visual reminder that model complexity must match the signal in the data. Too little complexity underfits; too much complexity overfits.

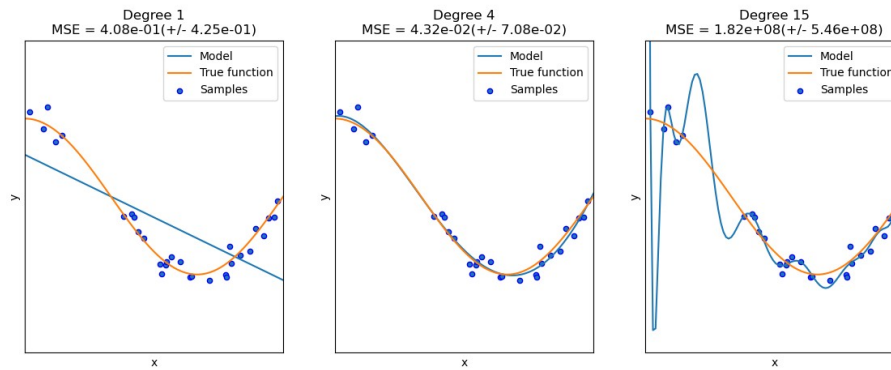


Figure 7. Underfitting and overfitting illustration. Model capacity that is too low misses structure, while capacity that is too high can chase noise instead of the true trend. Source: scikit-learn developers [22].

Figure 8 adds another practical view: learning curves compare training and validation performance as more data are used. When both scores are poor, the model may be underfit or the features may be weak. When the training score is strong but the validation score is weak, the model may be overfit [22], [23].

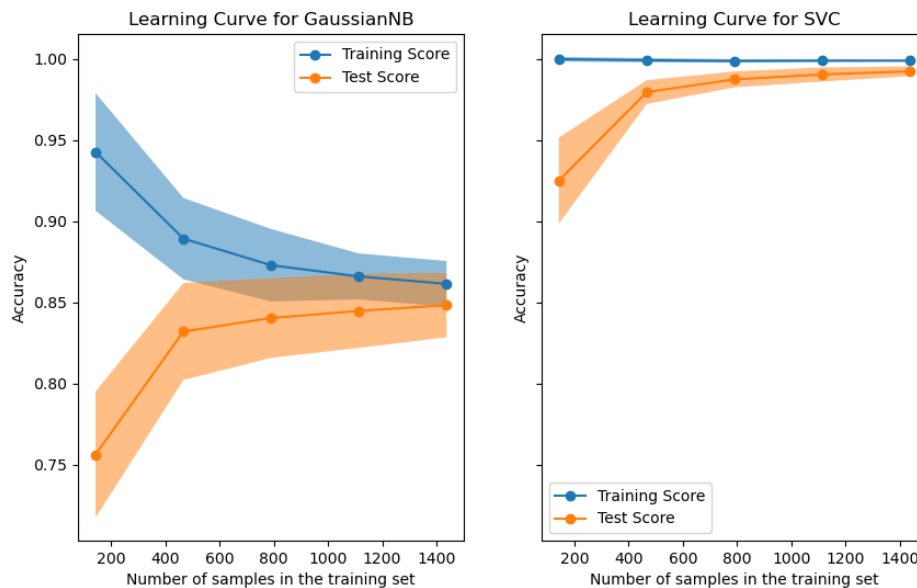


Figure 8. Learning curves showing the relationship between training score and validation score as more training examples are added. Source: scikit-learn developers [23].

8.2 Learning-rate problems

The learning rate controls the size of each optimization step. If the learning rate is too large, the optimizer may jump back and forth across a good region instead of settling into it. If it is too small, learning can become extremely slow. In gradient-based training, the update rule can be written as a current parameter value minus a learning-rate multiplier times the gradient of the loss.

$$\theta_{(t+1)} = \theta_t - \eta \nabla J(\theta_t)$$

Here θ represents the parameters, η is the learning rate, and $\nabla J(\theta_t)$ is the gradient of the loss at step t . Large η can cause divergence or oscillation. Very small η can make learning appear stuck.

Inputs and outputs. X has shape (800, 6), y is a continuous target, and each fit call returns a history object containing training and validation loss and metric values over epochs.

```
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split

# Synthetic material-strength style regression data
rng = np.random.default_rng(42)
X = rng.normal(size=(800, 6))
y = (
    30
    + 4.0 * X[:, 0]
    - 2.0 * X[:, 1]
    + 1.5 * X[:, 2] * X[:, 3]
    + rng.normal(0, 1.2, size=800)
)

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.20, random_state=42
)

def build_model(learning_rate):
    model = tf.keras.Sequential(
        [
            tf.keras.layers.Dense(32, activation="relu", input_shape=(6,)),
            tf.keras.layers.Dense(32, activation="relu"),
            tf.keras.layers.Dense(1),
        ]
    )
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
        loss="mse",
        metrics=["mae"],
    )
    return model

bad_model = build_model(learning_rate=0.10)
good_model = build_model(learning_rate=0.001)

bad_history = bad_model.fit(
    X_train, y_train,
    validation_data=(X_valid, y_valid),
    epochs=50,
    verbose=0,
)

good_history = good_model.fit(
    X_train, y_train,
    validation_data=(X_valid, y_valid),
    epochs=50,
    verbose=0,
)

print("final bad validation MAE:", bad_history.history["val_mae"][-1])
print("final good validation MAE:", good_history.history["val_mae"][-1])
```

Educational purpose. The code compares an obviously large learning rate with a more reasonable one so the reader can see how training stability changes.

The plot is a diagnostic tool. Divergence, oscillation, or a steadily widening train-validation gap should trigger further investigation.

```
import matplotlib.pyplot as plt

plt.plot(bad_history.history["loss"], label="bad train")
plt.plot(bad_history.history["val_loss"], label="bad valid")
plt.plot(good_history.history["loss"], label="good train")
plt.plot(good_history.history["val_loss"], label="good valid")
plt.xlabel("Epoch")
plt.ylabel("MSE loss")
plt.legend()
plt.show()
```

8.3 Batch size, initialization, and unstable validation

Batch size controls how many examples are used to estimate the gradient at each update. Very small batches can make the optimization path noisy, while very large batches can reduce the natural stochasticity that sometimes helps optimization explore. Initialization matters because poor starting values can slow learning or produce unstable early behavior. Validation behavior may also look unstable when the validation set is too small, highly noisy, or not representative of the operating conditions.

A useful rule for beginners is to change one factor at a time. First inspect the learning rate. Then inspect model size, batch size, and validation design. If many things are changed at once, the cause of instability becomes hard to diagnose.

Figure 9 shows how validation performance can change as a hyperparameter varies. This is a useful way to separate underfitting, overfitting, and a reasonable middle region.

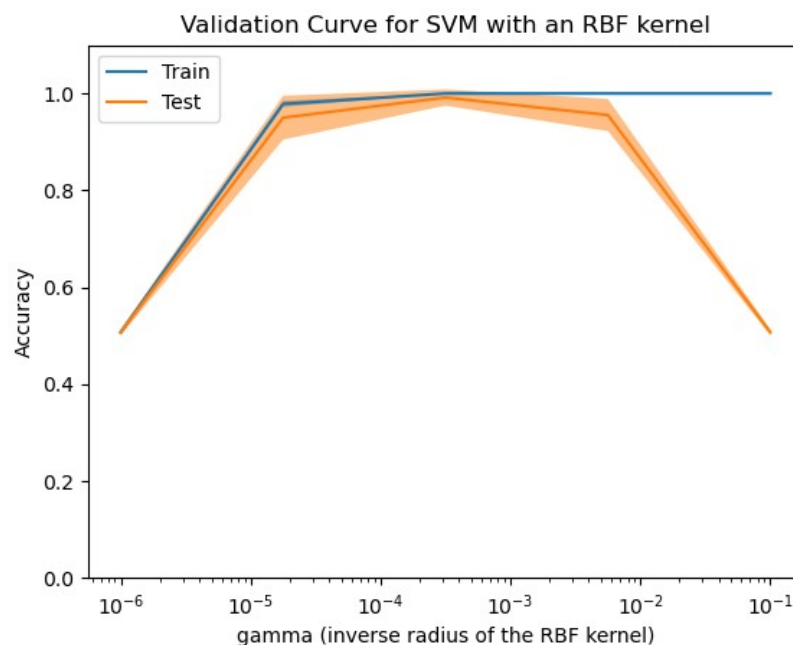


Figure 9. Validation-curve example. Validation curves are useful for diagnosing whether a model is underfit, overfit, or reasonably tuned. Source: scikit-learn developers [24].

8.4 Overfitting and underfitting

Underfitting means the model is too simple or too constrained to represent the important relationships in the data. Overfitting means the model starts to memorize noise or idiosyncrasies of the training data rather than the underlying pattern. In training curves, underfitting often appears as both training and validation errors staying high. Overfitting often appears as training error continuing to improve while validation error stops improving or starts worsening.

Symptom in curves	Likely cause	First action to try
Training and validation both poor	Underfitting, weak features, or too much regularization	Increase model capacity slightly or improve features.
Training good, validation poor	Overfitting	Use early stopping, regularization, or more data.
Loss diverges or oscillates	Learning rate too large or unstable input scaling	Reduce learning rate and inspect preprocessing.
Learning very slow	Learning rate too small or poor initialization	Increase learning rate moderately or inspect optimizer settings.
Validation highly erratic	Small or nonrepresentative validation set, noisy labels, or drift	Revisit split design and data quality.

8.5 Section summary

- Unstable training shows up in curves before it shows up in final disappointment.
- Learning-rate choice is one of the first things to inspect.
- Batch size, initialization, validation design, and model complexity all influence stability.
- Overfitting and underfitting can often be diagnosed visually from the gap between training and validation behavior.

9. Stabilization tools

Once unstable behavior has been diagnosed, the next step is to use simple stabilization tools. The emphasis here is on transparent, beginner-friendly methods that work well in common neural-network workflows: learning-rate adjustment, scheduled reduction, early stopping, L2 regularization, dropout, and validation monitoring [11]-[13].

9.1 Learning-rate adjustment and scheduled reduction

Purpose. Learning-rate adjustment changes the size of the optimization steps so that the model can learn efficiently without overshooting. A scheduled reduction lowers the learning rate when progress stalls. Inputs and outputs. The input is the optimizer state plus recent training or validation behavior. The output is a revised learning rate that often allows smoother convergence.

Why it matters in Mechanical Engineering. Field datasets can be noisy, and the useful signal may be subtle. In that situation, a learning rate that is too aggressive can make the model bounce around instead of extracting the underlying relationship.

9.2 Early stopping

Early stopping monitors a validation metric and stops training when the model has stopped making meaningful progress. Its purpose is to reduce unnecessary training and prevent the model from continuing deeper into overfitting. The key inputs are the monitored quantity, such as validation loss, and patience, which specifies how many epochs without improvement are tolerated [11].

Interpretation. Early stopping is not a magic fix. It works best when the validation set is representative and the monitored metric matches the real engineering objective.

9.3 Regularization and L2 penalties

Regularization discourages overly complex parameter values. L2 regularization adds a penalty proportional to the sum of squared weights. This nudges the model toward smaller weights and often improves generalization.

$$J(\theta) = L_{\text{data}}(\theta) + \lambda \sum_j w_j^2$$

Here L_{data} is the original data-fitting loss, λ controls the regularization strength, and w_j are the model weights. A larger λ pushes the weights toward smaller values. If λ is too large, the model may underfit. If λ is too small, the model may still overfit.

9.4 Dropout and validation monitoring

Dropout temporarily removes a random fraction of units during training. Its purpose is to reduce co-adaptation and encourage the network to rely on distributed patterns rather than narrow memorization [13]. Validation monitoring means reading the training history every few epochs instead of trusting the final epoch blindly. The input is the training history, and the output is a diagnostic judgment: continue, stop, or revise the design.

Tool	Purpose	Typical beginner setting	When it helps most
Learning-rate	Make late-stage updates	Monitor validation loss and	Loss stops improving

reduction	smaller	reduce when it plateaus	smoothly.
Early stopping	Stop before overfitting grows	patience of a few epochs	Validation performance peaks early.
L2 regularization	Diwearage very large weights	Small λ such as 1e-4 or 1e-3	Model is too flexible for the data.
Dropout	Reduce memorization of narrow patterns	0.1 to 0.5 depending on model size	Training strongly outperforms validation.
Validation monitoring	Read curves continuously	Inspect loss and key metrics together	Any sign of unstable or misleading training.

9.5 A practical Keras template for stable training

Inputs and outputs. The model takes tabular features with shape (n_samples, 6) and produces one continuous prediction per sample. The callbacks monitor validation loss and adjust training automatically.

```
import tensorflow as tf

stable_model = tf.keras.Sequential(
    [
        tf.keras.layers.Input(shape=(6,)),
        tf.keras.layers.Dense(
            64,
            activation="relu",
            kernel_regularizer=tf.keras.regularizers.l2(1e-4),
        ),
        tf.keras.layers.Dropout(0.20),
        tf.keras.layers.Dense(
            32,
            activation="relu",
            kernel_regularizer=tf.keras.regularizers.l2(1e-4),
        ),
        tf.keras.layers.Dense(1),
    ]
)

stable_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="mse",
    metrics=["mae"],
)

callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=10,
        restore_best_weights=True,
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=5,
        min_lr=1e-5,
    ),
]
```

```

history = stable_model.fit(
    X_train,
    y_train,
    validation_data=(X_valid, y_valid),
    epochs=200,
    batch_size=32,
    callbacks=callbacks,
    verbose=0,
)

```

This compact template includes four stabilization ideas at once: a moderate learning rate, L2 regularization, dropout, and validation-based callbacks.

- Input layer: defines the number of input features.
- Dense layers: learn nonlinear relationships between the Mechanical Engineering features and the target.
- `kernel_regularizer=tf.keras.regularizers.l2(1e-4)`: adds an L2 penalty to the layer weights.
- `Dropout(0.20)`: randomly disables 20 percent of the units during training.
- `Adam(learning_rate=1e-3)`: starts with a moderate learning rate instead of an aggressive one.
- `EarlyStopping(... restore_best_weights=True)`: stops training after the validation loss has stopped improving and returns the best seen weights.
- `ReduceLROnPlateau(...)`: lowers the learning rate automatically if validation loss stalls.

9.6 Section summary

- Learning-rate control is one of the most effective first tools for improving stability.
- Early stopping and validation monitoring help prevent unnecessary over-training.
- L2 regularization and dropout can improve generalization when the model is too flexible.
- A small number of well-understood tools is better for beginners than a large number of poorly understood tweaks.

10. Uncertainty communication and risk-aware decision-making

A model output is not the same thing as a decision. The model may produce a probability or a point prediction, but an engineer still has to decide what action to take. Uncertainty communication helps the analyst explain how much confidence the model has, how that confidence should be interpreted, and what error range should be expected in practice [3], [9], [10].

10.1 Hard decisions versus confidence levels

Plain-language explanation. A hard decision is a yes or no output after applying a threshold. A confidence level is the model's estimated probability or score before that threshold is applied. For example, a defect probability of 0.95 and a defect probability of 0.55 may both produce the decision 'inspect', but they should not be communicated as equally certain.

Mechanical Engineering interpretation. In a safety-related screening system, a 95 percent confidence level may justify immediate inspection, while a 55 percent confidence level may justify re-imaging, human review, or lower-priority follow-up. The key idea is that probability supports triage and prioritization, not only binary labeling.

Inputs and outputs. The input is a vector of predicted probabilities. The output is a threshold-versus-metric table that helps choose a decision rule.

```
import numpy as np
import pandas as pd
from sklearn.metrics import precision_score, recall_score, f1_score

proba = weighted_model.predict_proba(X_valid)[: , 1]
thresholds = np.linspace(0.10, 0.90, 9)

rows = []
for t in thresholds:
    pred_t = (proba >= t).astype(int)
    rows.append(
        {
            "threshold": round(t, 2),
            "precision": round(precision_score(y_valid, pred_t), 3),
            "recall": round(recall_score(y_valid, pred_t), 3),
            "f1": round(f1_score(y_valid, pred_t), 3),
        }
    )

threshold_table = pd.DataFrame(rows)
print(threshold_table)
```

10.2 Probability calibration and reliability diagrams

A calibrated classifier is one whose stated probabilities match observed frequencies. If the model repeatedly assigns around 0.80 probability to a defect, then roughly 80 percent of those cases should actually contain defects. Calibration matters because poorly calibrated probabilities can make a model sound more certain than it really is [9], [10].

Figure 10 combines two useful views: a reliability diagram and a confidence histogram. Together they show whether the predicted probabilities are meaningful and where the model is placing its confidence.

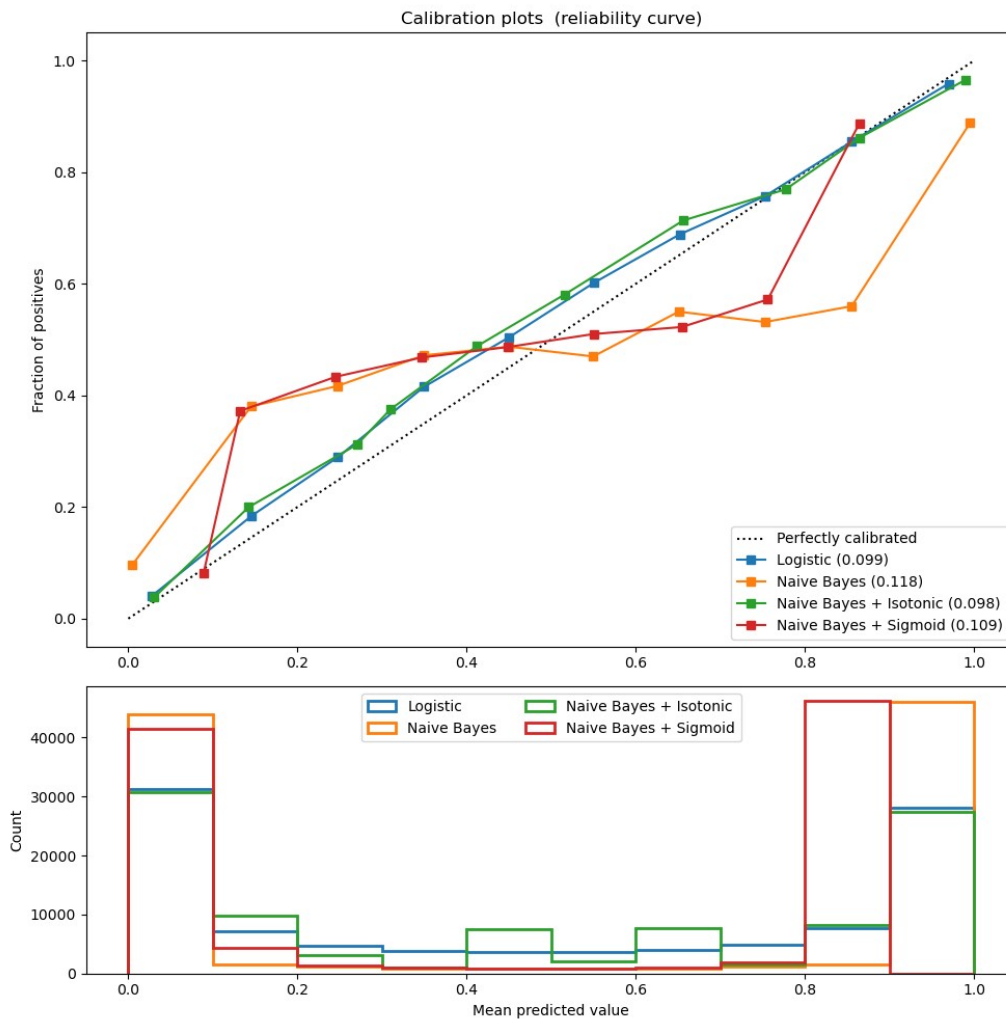


Figure 10. Probability calibration curve with a confidence histogram. A well-calibrated classifier reports probabilities that match observed frequencies. Source: scikit-learn developers [25].

The reliability diagram compares predicted confidence with observed frequency. A perfectly calibrated model lies close to the diagonal reference line.

```
from sklearn.calibration import CalibrationDisplay

CalibrationDisplay.from_predictions(
    y_valid,
    proba,
    n_bins=10,
)
plt.show()
```

10.3 Decision thresholds and threshold-versus-metric thinking

Threshold choice is a policy decision informed by model behavior. Lower thresholds usually increase recall and catch more positives, but they also tend to reduce precision by increasing false positives. Higher thresholds do the opposite. There is no universal best threshold. The appropriate threshold depends on the consequence of missed detections versus false alarms.

This plot turns threshold selection into an engineering discussion instead of a guess.

```
plt.plot(threshold_table["threshold"], threshold_table["precision"],
label="precision")
plt.plot(threshold_table["threshold"], threshold_table["recall"], label="recall")
plt.plot(threshold_table["threshold"], threshold_table["f1"], label="F1")
plt.xlabel("Decision threshold")
plt.ylabel("Metric value")
plt.legend()
plt.show()
```

10.4 Prediction intervals and residual-based error bands

For regression, a single point prediction hides the spread of likely error. A prediction interval is a range around the predicted value within which future observations are expected to fall with a chosen empirical frequency. A simple beginner-friendly way to build such intervals is to use the distribution of residuals on validation data.

$$e_i = y_i - \hat{y}_i$$

$$\text{empirical interval: } \hat{y}_{\text{new}} + [q_{\text{low}}(e), q_{\text{high}}(e)]$$

Figure 11 shows why an interval can communicate risk more honestly than a single predicted value.

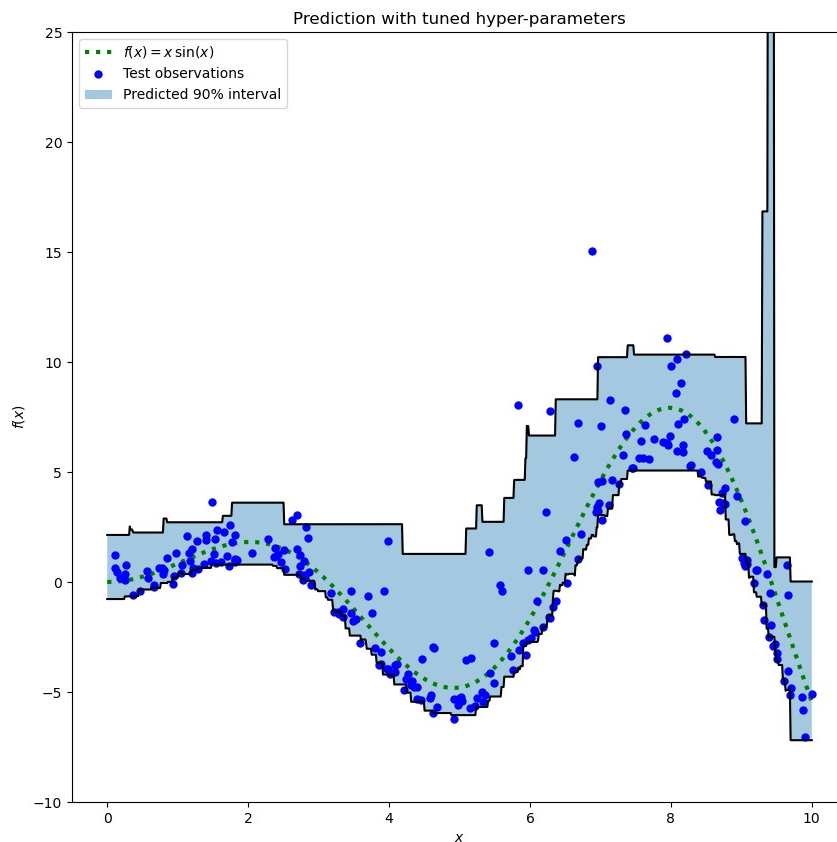


Figure 11. Example prediction intervals in regression. Intervals communicate that a point prediction is not the whole story, especially in noisy field data. Source: scikit-learn developers [27].

Inputs and outputs. The input is a trained regression model and a set of validation residuals. The output is a point prediction plus a lower and upper error band.

```
from sklearn.metrics import mean_absolute_error

# Residual-based empirical interval
valid_pred = m1.predict(X_test)
residuals = y_test - valid_pred

low_q, high_q = np.percentile(residuals, [5, 95])

new_pred = m1.predict(X_test[:3])
lower_band = new_pred + low_q
upper_band = new_pred + high_q

for pred, low, high in zip(new_pred, lower_band, upper_band):
    print(
        "prediction =", round(pred, 2),
        "interval = [", round(low, 2), ",", round(high, 2), "]",
    )
```

10.5 Repeated-run uncertainty summaries

A second practical tool is to retrain the same model several times with different random seeds and summarize how much the predictions vary. This does not capture every form of uncertainty, but it provides a useful stability check for beginners. Large run-to-run variation suggests the model or the training setup is sensitive.

Interpretation. If the repeated-run band is wide, the model is sensitive to initialization or data order and its outputs should be reported with caution.

```
all_predictions = []

for seed in [1, 2, 3, 4, 5]:
    tf.keras.utils.set_random_seed(seed)
    model = build_model(learning_rate=0.001)
    model.fit(X_train, y_train, epochs=50, verbose=0)
    all_predictions.append(model.predict(X_valid[:20], verbose=0).ravel())

pred_matrix = np.vstack(all_predictions)
mean_pred = pred_matrix.mean(axis=0)
low_pred = np.percentile(pred_matrix, 5, axis=0)
high_pred = np.percentile(pred_matrix, 95, axis=0)

print("first mean prediction:", round(mean_pred[0], 3))
print("first 90% repeated-run band:", round(low_pred[0], 3), round(high_pred[0], 3))
```

10.6 Communicating uncertainty honestly

Responsible communication should answer four questions: what the model predicts, how confident the model is, how that confidence was checked, and what action is recommended at the chosen threshold. In practice, a good engineering summary often includes a point prediction or probability, a threshold-based interpretation, a statement about calibration or

residual error, and a reminder that the model supports rather than replaces engineering judgment.

Method	Output shown to decision-maker	Best use
Probability with threshold	Example: 0.78 probability, inspect above 0.70	Binary screening and triage
Calibration statement	Example: probabilities were checked against observed frequencies	When confidence values will be acted on directly
Prediction interval	Example: strength = 36 MPa, likely range 32 to 40 MPa	Continuous predictions with meaningful uncertainty
Repeated-run summary	Example: prediction stable across five random restarts	Checking model sensitivity and robustness

Honest wording example. The model estimates a 0.82 probability of significant distress. Using the current screening policy, this exceeds the inspection threshold of 0.70. The probabilities were checked with a calibration plot, but performance should be reviewed again after new site data are collected.

10.7 Section summary

- A model decision is not the same as a model confidence level.
- Calibration checks whether predicted probabilities match observed frequencies.
- Thresholds should be chosen based on engineering consequences, not only convenience.
- Prediction intervals and repeated-run summaries provide practical ways to communicate uncertainty in regression and model stability.

11. Theory and mathematics for training stability and uncertainty

This section consolidates the main mathematical ideas from Part II. The purpose is not to make the reader derive optimization theory from first principles. The purpose is to connect the symbols seen in code to clear engineering interpretation.

11.1 Learning-rate effects on optimization

In gradient-based training, the optimizer moves the parameters in the direction that reduces loss. The learning rate controls the step length. Large steps may overshoot useful regions. Small steps may barely move. A moderate learning rate often gives the most stable progress.

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

11.2 Generalization and overfitting intuition

Generalization means the model performs well on new data, not only on the examples it already saw during training. Overfitting occurs when the model captures noise or accidental details of the training set. This is often visible as a widening gap between training and validation performance.

11.3 Regularization penalties

L2 regularization modifies the training objective by adding a penalty on large weights. This can smooth the learned function and reduce sensitivity to noise. The strength parameter λ must be chosen carefully because too much regularization can suppress useful structure.

$$J(\theta) = L_{\text{data}}(\theta) + \lambda \sum_j w_j^2$$

11.4 Calibration intuition and threshold logic

Calibration asks whether probabilities correspond to observed frequencies. Threshold logic asks how a probability is converted into an action. These are different questions. A model may be good at ranking high-risk cases ahead of low-risk cases while still being poorly calibrated in its probability values. A threshold then overlays a decision policy on top of those probabilities.

$$\text{predict positive if } P(y = 1 | x) \geq t$$

11.5 Empirical prediction intervals and error bands

If the residuals on validation data are stored, the percentiles of those residuals can be used to create empirical error bands around future predictions. This approach is simple, transparent, and useful for classroom projects. It should still be presented as an empirical approximation rather than a guarantee.

$$\text{lower band} = \hat{y} + q_{\text{low}}(\text{residuals})$$

$$\text{upper band} = \hat{y} + q_{\text{high}}(\text{residuals})$$

Idea	Compact formula	Interpretation
Optimization update	$\theta_{(t+1)} = \theta_t - \eta \nabla J(\theta_t)$	Learning rate controls the step size during training.
L2 regularization	$J = L_{\text{data}} + \lambda \sum w_j^2$	Large weights are penalized.
Threshold rule	predict positive if $p \geq t$	Decision policy is separated from the probability itself.
Residual interval	$\hat{y} + [q_{\text{low}}, q_{\text{high}}]$	Point prediction is expanded into an error band.

11.6 Section summary

- Training stability depends strongly on the size of the optimization step.
- Generalization is about future performance, not only training performance.
- Regularization and threshold choice each change the behavior of the final model in interpretable ways.
- Empirical uncertainty summaries are practical tools for communicating model risk.

12. Integrated Mechanical Engineering mini-workflows

This section ties the earlier ideas together into concise workflows. The goal is to show how a beginner can move from a dataset to a responsible modeling process without skipping the hard questions.

12.1 Rare defect detection in mechanical systems inspection

- Start by counting how many inspection images or records are labeled defect and no defect.
- Use a confusion matrix, precision, recall, and F1-score rather than accuracy alone.
- Try a baseline classifier first, then compare class weighting, oversampling, undersampling, and SMOTE on the training data only.
- Review threshold-versus-metric curves to choose an inspection trigger that matches the risk of missed defects.
- Report confidence levels with the final inspection recommendation rather than only binary labels.

Typical output. A transparent screening report might state: 94 percent recall at a 0.40 threshold, moderate precision, and a recommendation that all cases above 0.70 probability receive immediate engineer review.

12.2 Mechanical material strength prediction

- Inspect mixture variables for missing values, impossible unit ranges, and outliers before fitting any regression model.
- Use mean or median imputation for a simple baseline and compare the model before and after outlier handling.
- Train a modest regression model and monitor validation error rather than only training error.
- Add a residual-based prediction interval so the output is not presented as a single exact strength value.
- When unusual mixtures appear, communicate that the prediction is based on limited comparable training examples.

12.3 Structural health monitoring from sensor data

- Respect time order when splitting training, validation, and test data.
- Use rolling statistics and simple drift checks to detect sensor baseline changes over time.
- Treat missingness during severe events as potentially informative rather than automatically harmless.
- Inspect training stability carefully because noisy monitoring signals can produce erratic validation behavior.
- When reporting anomalies, combine predicted score, recent drift status, and model confidence.

12.4 Mechanical System Reliability and Safety: site and time effects

Mechanical System Reliability and Safety often requires reasoning across places and across time. A model trained on one site may face different material, land-cover, coolant return, or exposure conditions at another site. A model trained on one climate regime may later be used during a different hazard season. This makes drift checks, external validation, and honest uncertainty communication especially important.

- Track site identity and collection period as important context variables.
- Compare feature distributions across sites before transferring a model.
- Recheck threshold policies when the consequence of false negatives becomes more severe under hazard conditions.
- Use intervals or confidence categories when communicating results to resilience planners and asset managers.
- Plan a retraining review if both the data distribution and recent performance indicate meaningful change.

12.5 Deployment checklist

Stage	Checklist item	Why it matters
Before training	Count classes, inspect missingness, flag extreme values	Prevents blind trust in raw data.
During training	Monitor training and validation curves	Catches divergence and overfitting early.
Before deployment	Check calibration or residual spread	Prevents overconfident communication.
During deployment	Track drift and recent performance	Helps detect when the model is leaving its comfort zone.
After deployment review	Document updates, thresholds, and limitations	Creates traceable engineering accountability.

12.6 Section summary

- The earlier ideas become more useful when organized into repeatable workflows.
- Different Mechanical Engineering applications share the same core diagnostic questions even when the data type changes.
- Deployment is not the end of the process; drift and uncertainty must continue to be monitored.

13. Best practices and common mistakes

Beginners often make errors not because the mathematics is too advanced, but because the workflow is rushed. This section turns the main lessons of the tutorial into a short list of do's and don'ts.

13.1 A practical checklist before, during, and after training

Phase	Best practice	Reason
Before training	Inspect class counts, missing values, units, and extreme values	Data problems are easier to fix before modeling begins.
Before training	Set aside validation and test data early	Prevents leakage and optimistic scoring.
During training	Monitor both training and validation curves	Detects instability, overfitting, and slow learning.
During training	Change one major setting at a time	Improves diagnosis of cause and effect.
After training	Check calibration or residual spread	Helps communicate confidence honestly.
After training	Document thresholds, fixes, and limitations	Supports transparent engineering use.

13.2 Common mistakes made by beginners

- Trusting accuracy alone on an imbalanced dataset.
- Imputing or resampling before splitting the data, which leaks information across train and validation boundaries.
- Deleting all outliers automatically without checking whether they represent real engineering extremes.
- Ignoring site and time effects when moving a model to a new environment.
- Reading only the last epoch instead of reading the whole training and validation history.

- Using predicted probabilities as if they were perfectly calibrated without checking them.
- Reporting a single point prediction with no uncertainty statement in a risk-sensitive engineering setting.

Professional habit. Whenever a fix is applied, write down three things: what was changed, why it was changed, and how performance changed afterward.

13.3 Section summary

- Good practice is usually a matter of disciplined workflow, not exotic methods.
- Transparent documentation is part of model reliability.
- A responsible machine learning result in Mechanical Engineering always includes limitations and uncertainty.

14. Conclusion

Machine learning becomes much more useful in Mechanical Engineering when it is treated as a diagnostic and decision-support tool rather than a black-box score generator. Real engineering data are messy. They are imbalanced, incomplete, noisy, and time-varying. Training can be unstable. Probabilities can be overconfident. For that reason, the quality of a project depends as much on diagnosis and communication as on the choice of model.

For beginners, the strongest path is usually the simplest one: inspect the data carefully, use transparent fixes, monitor training honestly, and report uncertainty in a form that decision-makers can understand. When these habits are combined with Mechanical Engineering judgment, machine learning can support more robust, transparent, and responsible decisions in materials, structures, hazards, monitoring, and mechanical systems management.

References

1. Cha, Y.-J., et al. (2024). Deep learning-based structural health monitoring. *Automation in Construction*, 161, 105328. <https://doi.org/10.1016/j.autcon.2024.105328>
2. Vadyala, S. R., Betgeri, S. N., Sheri, A. M., & Devarakonda, A. K. (2022). A review of physics-based machine learning in mechanical engineering. *Results in Engineering*, 13, 100316. <https://doi.org/10.1016/j.rineng.2021.100316>
3. Wang, Q., et al. (2024). Machine learning aided uncertainty quantification for engineering structures involving material-geometric randomness and data imperfection. *Computer Methods in Applied Mechanics and Engineering*, 423, 116868. <https://doi.org/10.1016/j.cma.2024.116868>
4. Uddameri, V., & Hernandez, E. A. (2025). Machine Learning for Cooling-system overload Resiliency: Current Status and Unexplored Directions. *Environments*, 12(8), 259. <https://doi.org/10.3390/environments12080259>
5. Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3), 581-592.
6. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
7. He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263-1284.
8. Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLoS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
9. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70, 1321-1330.
10. Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of the 22nd International Conference on Machine Learning*, 625-632.
11. TensorFlow. (2026). EarlyStopping. Keras API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping
12. TensorFlow. (2026). ReduceLROnPlateau. Keras API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/ReduceLROnPlateau
13. TensorFlow. (2026). Dropout layer. Keras API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dropout
14. scikit-learn developers. (2026). SimpleImputer. scikit-learn API reference. <https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>
15. imbalanced-learn developers. (2026). SMOTE. imbalanced-learn API reference. https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html
16. scikit-learn developers. (2026). Probability calibration and CalibrationDisplay. scikit-learn documentation. <https://scikit-learn.org/stable/modules/calibration.html>
17. Feature Engineering and Selection. (2026). Missing data mechanisms (MCAR, MAR, and MNAR) figure. <https://feaz-book.com/diagrams/missing-mcar-mar-mnar.png>
18. scikit-learn developers. (2026). Confusion matrix example figure. https://scikit-learn.org/0.21/_images/sphx_glr_plot_confusion_matrix_002.png

19. scikit-learn developers. (2026). Precision-Recall example figure.
https://scikit-learn.org/stable/_images/sphx_glr_plot_precision_recall_001.png
20. imbalanced-learn developers. (2026). Sample generator used in SMOTE-like samplers figure.
https://imbalanced-learn.org/stable/_images/sphx_glr_plot_illustration_generation_sample_001.png
21. scikit-learn developers. (2026). Robust linear estimator fitting figure.
https://scikit-learn.org/1.5/_images/sphx_glr_plot_robust_fit_003.png
22. scikit-learn developers. (2026). Underfitting versus overfitting figure.
https://scikit-learn.org/stable/_images/sphx_glr_plot_underfitting_overfitting_001.png
23. scikit-learn developers. (2026). Plotting learning curves figure.
https://scikit-learn.org/stable/_images/sphx_glr_plot_learning_curve_001.png
24. scikit-learn developers. (2026). Plotting validation curves figure.
https://scikit-learn.org/1.5/_images/sphx_glr_plot_validation_curve_001.png
25. scikit-learn developers. (2026). Probability calibration curves figure.
https://scikit-learn.org/0.23/_images/sphx_glr_plot_calibration_curve_001.png
26. NannyML. (2026). Data Drift for Continuous Variables: K-S Test Explained.
<https://www.nannyml.com/blog/data-drift-for-continuous-variables>
27. scikit-learn developers. (2026). Prediction intervals for gradient boosting regression figure.
https://scikit-learn.org/1.5/_images/sphx_glr_plot_gradient_boosting_quantile_002.png